

Section	Easy	Medium	Expert
Part 1 — OOP	Easy: Q1–Q10	Medium: Q11–Q16	Expert: None
Part 2 — Android	Easy: Q17–Q31	Medium: Q32–Q62	Expert: Q63–Q77

PART 1 — Object-Oriented Programming Answers

Q1 – Q16

EASY — OOP Foundations (Q1 – Q10)

1 What is OOP? What are its four core principles?

OOP is a programming paradigm that organises code into objects combining data and behaviour. The four principles are:

Encapsulation — bundling data and methods, hiding internal state behind controlled access.

Inheritance — a class acquiring properties and behaviour of another class.

Polymorphism — one interface, multiple implementations; the same method call behaves differently depending on the object.

Abstraction — exposing only necessary functionality, hiding implementation complexity.

2 What is the difference between a class and an object?

A **class** is a blueprint or template — it defines structure and behaviour but occupies no memory of its own. An **object** is a live instance of a class — it occupies memory and holds actual state. Example: **Car** is the class; **myCar = new Car()** is the object.

3 What is encapsulation? How do access modifiers support it in Java?

Encapsulation hides internal state and requires all interaction through public methods (getters/setters). **private** prevents direct outside access to fields. **protected** allows access within the same package and subclasses. **public** exposes to everyone. Together they ensure data is only modified in validated, controlled ways.

4 What is polymorphism? Give a simple example of runtime polymorphism in Java.

Polymorphism means 'many forms' — the same method call behaves differently depending on the actual object type. Runtime polymorphism is resolved during program execution via dynamic dispatch:

```
Animal a = new Dog(); a.makeSound(); // calls Dog's makeSound(), not Animal's – resolved at runtime
```

5 What is abstraction? Abstract class vs concrete class? What is a constructor? Can a constructor be private?

Abstraction hides implementation details and shows only relevant functionality. An **abstract class** cannot be instantiated and may have abstract methods (no body) that subclasses must implement. A **concrete class** implements all methods and can be instantiated. A **constructor** is a special method invoked when an object is created to initialise its state. Yes, a constructor can be **private** — this is the basis of the Singleton pattern, preventing external instantiation.

6 What is method overloading? What is method overriding? Role of @Override?

Overloading = same method name, different parameter types/count — resolved at **compile time** (static polymorphism).

Overriding = subclass redefines a parent method with the exact same signature — resolved at **runtime** via dynamic dispatch. **@Override** is a compile-time annotation that causes a compile error if the method does not actually override anything — prevents silent bugs from typos in method names.

7 What is an interface in Java? How is it different from an abstract class?

	Interface	Abstract Class
Instantiation	Cannot	Cannot
State (fields)	Only static final	Can have instance fields
Multiple inheritance	Yes (multiple)	No (single only)
Methods	All abstract; default/static (Java 8+)	Mix of abstract + concrete
Constructor	None	Can have

Use an **interface** for a contract across unrelated classes. Use an **abstract class** when sharing common state or partial implementation.

8 Procedural vs OOP? What does the `final` keyword do?

Procedural: top-down execution, functions act on global data, no encapsulation, harder to reuse. **OOP**: data and behaviour bundled in objects, promotes encapsulation, inheritance, and reuse.

final on a **field**: cannot be reassigned after initialisation. On a **method**: cannot be overridden by subclasses. On a **class**: cannot be subclassed (e.g., `String` is final in Java).

9 What is an `enum` in Java? Is it a class? Can it have methods?

Yes, an **enum** is a special class in Java representing a fixed set of named constants. Each constant is a static final instance of the enum type. It can have constructors, fields, and methods:

```
enum Direction { NORTH, SOUTH, EAST, WEST; public boolean isVertical() { return this == NORTH || this == SOUTH; } }
```

10 What is the difference between `==` and `.equals()` in Java?

`==` compares **references** (memory addresses) — checks if two variables point to the exact same object. `.equals()` compares **content/value**. For `String`, `==` may accidentally work for literals (due to String pool interning) but fails for dynamically created strings. Always use `.equals()` for String comparison. In Android, use `TextUtils.equals()` for null-safe comparison.

MEDIUM — OOP In Depth (Q11 – Q16)

11 Types of inheritance in Java? Why no multiple inheritance through classes?

Single: one parent. **Multilevel**: $A \rightarrow B \rightarrow C$. **Hierarchical**: $A \rightarrow B$, $A \rightarrow C$ (multiple subclasses from one parent). **Multiple**: Java does NOT support this via classes because of the **diamond problem** — if B and C both extend A and override the same method, D extending both creates ambiguity about which version to call. Java solves this via **interfaces** (with explicit override required when conflict exists).

12 Diamond problem? How does Java resolve it with interface default methods?

The diamond problem: class D inherits from B and C, both of which have the same default method inherited from a common ancestor A. Java requires D to explicitly **override** the conflicting method, otherwise a compile-time error is thrown. Resolution rule: **class implementation > interface default > parent interface default**. The compiler forces you to resolve ambiguity explicitly.

13 Association, aggregation, and composition? has-a vs is-a?

Association: any general relationship between objects (weakest). **Aggregation:** 'has-a' where the child can exist independently — a Department has Employees, but Employees exist without Department. **Composition:** strong 'has-a' where child cannot exist without parent — a House has Rooms; Rooms don't exist without the House. **is-a** = inheritance relationship. **has-a** = composition or aggregation.

14 Composition vs inheritance? When to prefer composition?

Prefer **composition** when: the relationship is has-a not is-a, when you want reuse without committing to a hierarchy, when you need to swap behaviour at runtime, or when the parent class might change (fragile base class problem). Deep inheritance hierarchies create tight coupling. **Favour composition over inheritance** (Gang of Four principle).

15 Coupling, cohesion? Explain the SOLID principles.

Cohesion: how focused a class is on a single purpose — **high cohesion** is good. **Coupling:** how much a class depends on others — **low coupling** is good.

S — Single Responsibility: a class should have only one reason to change.

O — Open/Closed: open for extension, closed for modification.

L — Liskov Substitution: a subclass must be substitutable for its parent without breaking the program.

I — Interface Segregation: don't force classes to implement methods they don't use.

D — Dependency Inversion: depend on abstractions (interfaces), not concrete implementations.

16 Singleton pattern? Thread-safe double-checked locking in Java?

Singleton ensures only one instance exists. Thread-safe with double-checked locking:

```
public class Singleton { private static volatile Singleton instance; private Singleton() {}  
public static Singleton getInstance() { if (instance == null) { // First check (no lock)  
    synchronized (Singleton.class) { if (instance == null) { // Second check (with lock) instance  
        = new Singleton(); } } } return instance; } }
```

volatile is critical — without it, another thread could see a partially constructed object due to CPU instruction reordering.

EASY — Android Fundamentals (Q17 – Q31)

17 What is Android? What is the Android architecture stack?

Android is an open-source mobile OS by Google based on the Linux kernel. Stack from bottom to top: **Linux Kernel** (drivers, memory, power) → **HAL** (Hardware Abstraction Layer) → **ART** (Android Runtime — compiles .dex bytecode) → **Native C/C++ Libraries** (OpenGL, SQLite, WebKit) → **Java API Framework** (Activity Manager, View System, Content Providers) → **System Apps** (Phone, Chrome, Contacts).

18 What is `AndroidManifest.xml`? Name five things declared in it.

The Manifest is the app's configuration file — the OS reads it before running any code. Five declarations: (1) App **package name**, (2) all **components** (Activities, Services, Receivers, Providers), (3) **permissions** the app requires (uses-permission), (4) **minimum and target SDK versions**, (5) app **icon, label, and theme**.

19 What is an Intent? Explicit vs implicit? Toast vs Snackbar?

Intent: a messaging object to request an action from another component. **Explicit**: directly names the target component — used within your own app (new Intent(this, DetailActivity.class)). **Implicit**: declares an action and lets the system find a match — used to open maps, browsers, dialers (Intent.ACTION_VIEW).

Toast: floating transient message, not interactive, not tied to a View. **Snackbar**: anchored to CoordinatorLayout, can have an action button (e.g., UNDO), preferred by Material Design.

20 What is `Context`? Three types and when to use each?

Context provides access to app resources, system services, and allows launching components. Three types: **Application Context** (getApplicationContext()) — lives for the entire app process, safe for singletons and DI. **Activity Context** (this inside Activity) — required for dialogs, layout inflation with correct theme. **Service Context** — tied to Service lifecycle. Never hold an Activity context in a long-lived object — causes memory leaks.

21 Explain the Activity lifecycle. What is a Fragment? Add vs Replace?

Activity lifecycle: **onCreate** → **onStart** → **onResume** (interactive) → **onPause** → **onStop** → **onDestroy**. Also **onRestart** when resuming from stopped state.

Fragment: a modular, reusable UI component hosted inside an Activity. **Add**: places Fragment on top — the previous Fragment stays in memory and back stack (used for overlapping layers). **Replace**: removes previous Fragment and puts the new one in — standard navigation pattern.

22 Activity vs Fragment?

Activity is a full screen managed by the OS, must be declared in Manifest, heavier to create. Fragment is a reusable UI portion hosted inside an Activity, not declared in Manifest. Multiple Fragments can share one Activity. Fragments cannot exist without a host Activity. Fragments have a separate **View lifecycle** (onCreateView → onDestroyView) independent of their own lifecycle.

23 What is `ViewBinding`? How is it different from `findViewById`?

ViewBinding generates a typed binding class for each layout XML at compile time. Advantages over findViewById(): **null-safe** (compile error if ID doesn't exist), **type-safe** (no casting required), **faster** (no runtime view tree traversal). Always null the binding reference in **onDestroyView()** to prevent memory leaks when the Fragment stays in the back stack.

24 RecyclerView vs ListView? ViewHolder? ViewModel? Why survives rotation?

RecyclerView forces ViewHolder, supports multiple LayoutManagers (grid, staggered, linear), has built-in item animations, efficiently recycles views via RecycledViewPool. ListView creates new views for every item — deprecated.

ViewHolder: holds child view references so findViewById() is called once per item, not every bind.

ViewModel survives rotation because it is stored in a **ViewModelStore** retained via NonConfigurationInstance across configuration changes. **onCleared()** is only called when the Activity is truly finishing (back press), not on rotation.

25 MVVM architecture? Room? @Entity, @Dao, @Database?

MVVM: View (Fragment/Activity) observes ViewModel → ViewModel exposes LiveData → calls Repository → fetches from Room or network. No View references in ViewModel.

Room: Jetpack SQLite abstraction. **@Entity**: maps a Java class to a DB table. **@Dao**: defines query methods — Room validates SQL at compile time via annotation processor. **@Database**: main access point holding all DAOs and migration definitions.

26 What is `LiveData`? How does lifecycle awareness prevent crashes?

LiveData is a lifecycle-aware observable. It only delivers updates to observers in **STARTED** or **RESUMED** state. If a Fragment is destroyed or in the background, updates are queued and delivered when it resumes. This prevents the common crash of trying to update a destroyed UI. No manual unsubscribe needed — the LifecycleOwner handles cleanup automatically.

27 Permissions for Camera, Gallery, Google Places? Show place on map?

Camera: android.permission.CAMERA. **Gallery**: READ_EXTERNAL_STORAGE (API ≤ 32) or READ_MEDIA_IMAGES (API 33+). **Places API**: no Android runtime permission — only a valid API key in Manifest. **Location**: ACCESS_FINE_LOCATION.

To show a place: get SupportMapFragment → getMapAsync() → in onMapReady() call googleMap.addMarker(new MarkerOptions().position(latLng).title(name)) and googleMap.moveCamera(CameraUpdateFactory.newLatLngZoom(latLng, 15f)).

28 What is `DiffUtil`? How does it improve RecyclerView performance?

DiffUtil calculates the difference between two lists using **Myers' diff algorithm** and dispatches the minimal set of updates to the adapter. Instead of notifyDataSetChanged() (which redraws everything with no animation), it dispatches only notifyItemInserted/Removed/Changed — giving smooth animations and significantly better performance for large lists.

29 notifyDataSetChanged() vs notifyItemChanged()? Deep Linking vs Dynamic Linking?

notifyDataSetChanged(): invalidates the entire list — all items re-bind, no animations, poor performance.

notifyItemChanged(pos): updates only one item with animation. Always prefer specific notify methods or DiffUtil.

Deep Linking: a URI scheme that opens a specific screen in your app (myapp://product/123). **Dynamic Linking**: Firebase Dynamic Links — a smart URL that opens the app if installed, redirects to Play Store if not, then opens the correct screen after first launch.

30 How to make an API call in Android? How to implement dark mode?

API calls: use Retrofit — define an interface with @GET/@POST annotations, build a Retrofit instance with base URL and converter (Gson/Moshi), return RxJava Single or Kotlin Flow. In RxJava: subscribeOn(Schedulers.io()), observeOn(AndroidSchedulers.mainThread()).

Dark mode: use `AppCompatActivity.setDefaultNightMode()` to toggle. Use `DayNight` theme. Define dark overrides in `values-night/themes.xml`. Never hardcode colours — always reference theme attributes like `?attr/colorPrimary` so they auto-adapt.

31 What is `ProGuard`? Minification, obfuscation, optimization?

Minification: removes unused classes, methods, and fields — reduces APK size. **Obfuscation:** renames classes/methods/fields to short meaningless names (a, b, c) — makes reverse engineering harder. **Optimization:** restructures bytecode for better runtime performance. **R8** (built into AGP) does all three in one pass and is faster and produces smaller output than classic ProGuard.

MEDIUM — Android Intermediate (Q32 – Q62)

32 Complete Fragment lifecycle? onCreateView vs onViewCreated? Why null binding?

Fragment lifecycle: **onAttach** → **onCreate** → **onCreateView** → **onViewCreated** → **onStart** → **onResume** → **onPause** → **onStop** → **onDestroyView** → **onDestroy** → **onDetach**.

onCreateView: inflate and return the root View — do not access child views here. **onViewCreated:** View is fully ready — set click listeners, observe LiveData, access `binding.xyz` here.

Null binding in onDestroyView(): a Fragment can remain in the back stack while its View is destroyed. If binding is not nulled, it holds a reference to the destroyed View hierarchy preventing GC — a classic memory leak.

33 Explain all Android launch modes with real-world examples.

standard: new instance every time. Multiple instances in back stack. Default for most screens.

singleTop: if the Activity is already at the TOP of the stack, reuse it (`onNewIntent()` called). New instance if not on top. Use case: notification opening the same details screen.

singleTask: only one instance in the task. If it exists anywhere, all Activities above it are cleared and `onNewIntent()` is called. Use case: home/main screen.

singleInstance: one instance in its own isolated task — no other Activities share that task. Use case: a system-level screen like a call screen.

34 A→B→C→D: navigate from D back to A finishing B and C. Which flag?

Use **FLAG_ACTIVITY_CLEAR_TOP** combined with **FLAG_ACTIVITY_SINGLE_TOP**:

```
Intent intent = new Intent(this, AActivity.class);
intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP | Intent.FLAG_ACTIVITY_SINGLE_TOP);
startActivity(intent);
```

This destroys B, C, D from the stack and brings A to the front, calling `onNewIntent()` on A if it already exists.

35 How does FLAG_ACTIVITY_CLEAR_TOP work?

When the target Activity (e.g., A) already exists in the back stack, all Activities on top of it (B, C, D) are **destroyed** and A is brought to the front. Combined with **SINGLE_TOP**: A is reused and `onNewIntent()` is called. Without **SINGLE_TOP**: A is destroyed and recreated.

36 How to finish the entire application from a deeply nested Activity?

```
finishAffinity(); // API 16+ – finishes this and all parent Activities in the same task
```

Alternatively, set **android:noHistory="true"** on intermediate activities in the Manifest so they auto-finish when navigated away from. Avoid `System.exit(0)` — it is not recommended as it does not trigger proper lifecycle callbacks.

37 Service vs IntentService vs JobIntentService? When to prefer each?

Service: runs on the main thread — you must manage background threading manually. For long-running foreground operations. **IntentService:** handles each request on a worker thread sequentially, auto-stops when done. Deprecated API 30. **JobIntentService:** uses JobScheduler on API 26+ (respects Doze mode) and falls back to IntentService on older APIs. For background work that must run on older devices. **Today:** prefer **WorkManager** for all deferrable background work.

38 Service persist after app killed? START_STICKY vs START_NOT_STICKY?

Use a **Foreground Service** (call `startForeground()` with a persistent notification). The OS gives foreground services high priority and rarely kills them.

START_STICKY: if killed by OS, the Service is restarted with a null Intent. Use for services that must always run (media player, location tracker). **START_NOT_STICKY:** if killed, OS does NOT restart it. Use for one-off tasks where restarting without the original data makes no sense.

39 Activity-to-Activity, Activity-to-Fragment, Fragment-to-Fragment communication?

Activity → **Activity:** `Intent.putExtra()` / `ActivityResultLauncher` for results.

Activity → **Fragment:** call `fragment.setArguments(bundle)` before attaching, or call public methods on the Fragment reference.

Fragment → **Activity:** define a callback interface in Fragment, implement in Activity, cast `getActivity()`.

Fragment → **Fragment (preferred):** use a **shared ViewModel** scoped to the Activity — new `ViewModelProvider(requireActivity())`. Both Fragments observe the same ViewModel. Never communicate Fragment-to-Fragment directly.

40 Handle network errors gracefully? Resource wrapper?

Wrap API responses in a sealed **Resource<T>** class with three states:

```
Resource.loading(null) // show ProgressBar
Resource.success(data) // show data
Resource.error(msg, null) // show error Snackbar / retry
```

ViewModel emits these via `MutableLiveData`. Fragment observes and switches UI accordingly. Also implement retry logic, connectivity check via `ConnectivityManager`, and timeout/error mapping in the repository.

41 AsyncTask internals? Cons? What to use instead?

AsyncTask runs `doInBackground()` on a background thread from a thread pool, then posts `onPostExecute()` to the main thread via Handler. **Cons:** (1) Memory leaks — non-static inner class holds implicit reference to Activity. (2) Broken on rotation — Activity recreated but task still running. (3) Serial by default from API 11. (4) Deprecated API 30. **Use instead:** **RxJava3** (Single/Observable) or **ExecutorService + Handler** for Java projects.

42 AsyncTask.execute() parallel or serial? What is JobScheduler?

From **API 11 onwards**, `execute()` runs tasks **serially** by default on a single background thread. To run in parallel: `executeOnExecutor(AsyncTask.THREAD_POOL_EXECUTOR)`.

JobScheduler (API 21+): schedules background work that runs when conditions are met (network available, charging, idle). The OS batches jobs for battery efficiency. Does not guarantee immediate execution. Suitable for deferrable, non-urgent tasks.

43 Background execution limits in Android Oreo (API 26)? Notification changes?

Background limits: apps in the background cannot start background Services. Use `startForegroundService()` and call `startForeground()` within 5 seconds. Use **WorkManager** or **JobScheduler** for deferred work.

Notification channels: all notifications must be assigned to a **NotificationChannel**. Without a channel, notifications are silently dropped on API 26+. Users control each channel's sound, vibration, and importance independently from Settings.

44 Broadcast Receiver restrictions in Oreo? Headless Fragment?

Broadcast restrictions: apps cannot register receivers for most implicit broadcasts in the Manifest — they are silently dropped. Instead: register programmatically in code, or use JobScheduler/WorkManager. Explicit broadcasts and a small whitelist still work in Manifest.

Headless Fragment: a Fragment with no UI (setRetainInstance(true)) used to retain data across config changes before ViewModel existed. This is now **deprecated** — use ViewModel instead.

45 Memory leaks in Android — 5 common causes and fixes.

1. Static reference to Context/Activity: static field holding an Activity → use Application context or WeakReference.

2. Non-static inner class: implicit reference to outer Activity → use static inner class + WeakReference.

3. Unregistered listeners/observers: not unregistering BroadcastReceiver or LiveData → unregister in onStop()/onDestroyView().

4. Unmanaged RxJava subscriptions: use CompositeDisposable.clear() in ViewModel.onCleared().

5. Bitmaps not recycled: large bitmaps held in memory → use Glide/Coil for lifecycle-aware loading; call bitmap.recycle() for manual bitmaps.

46 Ways to optimise Android app performance? Build Variants and Product Flavours?

Performance: (1) ViewHolder + DiffUtil in RecyclerView. (2) Lifecycle-aware image loading with Glide/Coil. (3) Move heavy work off main thread. (4) R8 minification on release. (5) Flatten layouts with ConstraintLayout. (6) Avoid overdraw. (7) Use SparseArray over HashMap<Integer, Object>. (8) Profile with Android Studio Profiler.

Build Variants: combination of Build Type (debug/release) + Product Flavour. **Product Flavours:** allow different app versions from same codebase — e.g., free vs paid, or different API endpoints per white-label client (ohio, ct, florida).

47 Multithreading in Android? Compare HandlerThread, ExecutorService, ThreadPoolExecutor, RxJava.

HandlerThread: a Thread with a Looper — post tasks via Handler. Good for serial background work needing a message queue.

ExecutorService / ThreadPoolExecutor: pool of threads for parallel work. Executors.newFixedThreadPool(4) creates 4 threads. Use for controlled parallel execution.

RxJava3: Schedulers.io() for I/O bound, Schedulers.computation() for CPU, AndroidSchedulers.mainThread() for UI. Best for chaining async operations with transformation and error handling.

48 Repository pattern? LiveData.postValue() vs setValue()?

Repository abstracts data sources from ViewModel. It decides whether to return Room cache or fetch from network — acts as single source of truth. ViewModel never calls Retrofit or Room directly.

setValue(): must be called on the **main thread**. **postValue():** safe to call from any **background thread** — posts the update to the main thread internally. In RxJava chains on Schedulers.io(), use postValue() or observeOn(mainThread()) then setValue().

49 MutableLiveData vs LiveData? Why the private Mutable + public pattern?

MutableLiveData exposes public setValue() and postValue(). **LiveData** is read-only — no public setter. The pattern of private MutableLiveData + public LiveData ensures **only the ViewModel** can push new values. Fragments and Activities can only observe — they cannot directly modify state, preserving unidirectional data flow in MVVM.

50 What is ViewModelFactory? When needed? How used with Hilt?

ViewModelFactory is required when your ViewModel has constructor parameters the default factory cannot provide — e.g., passing a `userId`. Without Hilt: implement `ViewModelProvider.Factory` and override `create()`. With Hilt: use `@HiltViewModel` + `@Inject` on the constructor — Hilt generates the factory automatically, no manual factory needed.

51 WorkManager? Types of requests? Minimum repeat interval?

WorkManager guarantees execution even if the app exits or device restarts. **OneTimeWorkRequest**: runs once, can be chained. **PeriodicWorkRequest**: repeats at minimum **15 minutes** interval (OS enforced). Constraints: `NetworkType.CONNECTED`, `requiresCharging()`, `requiresDeviceIdle()`. Returns `Result.success()`, `Result.failure()`, or `Result.retry()`.

52 WorkManager vs JobScheduler?

WorkManager is a higher-level abstraction built on JobScheduler (API 23+), AlarmManager, and BroadcastReceiver depending on API level. WorkManager adds: persistent task chaining, input/output data between tasks, constraint tracking, LiveData observable status, and guaranteed execution across process death. Use **WorkManager for almost everything today**. Direct JobScheduler only for very specific low-level scheduling requirements.

53 Why ConstraintLayout? Barriers, Guidelines, Groups, Chains?

ConstraintLayout allows complex flat layouts without nesting — improving rendering performance. **Guideline**: virtual fixed line (percentage or dp offset) to constrain views against. **Barrier**: adjusts dynamically based on the largest referenced view — useful for varying text lengths. **Group**: controls visibility of multiple views simultaneously with one reference. **Chain**: distributes space between views (`spread`, `packed`, `spread_inside`).

54 What is Scoped Storage?

Introduced in Android 10 (Q), mandatory from Android 11. Apps no longer have unrestricted external storage access. Each app gets an isolated sandbox. To access files outside your sandbox: use **MediaStore** API for photos/videos/audio, or **Storage Access Framework** (DocumentProvider) for arbitrary files. `READ_EXTERNAL_STORAGE` is no longer sufficient for other apps' files on API 30+.

55 How to handle opening an Activity from a notification?

Create a **PendingIntent** pointing to the target Activity, attach via `setContentIntent()`. Key flags: **FLAG_IMMUTABLE** (required API 31+). For the Intent: `FLAG_ACTIVITY_SINGLE_TOP` (avoid recreating if already on top), `FLAG_ACTIVITY_CLEAR_TOP` (clear back stack). Use **TaskStackBuilder** to define a proper back stack so the back button works correctly after notification tap.

56 Push notifications — foreground vs background behaviour?

Foreground: `onMessageReceived()` in `FirebaseMessagingService` is called. You must **manually build and show** the notification — FCM does not auto-display it.

Background/killed: FCM automatically displays the notification from the **notification payload**. Tapping opens the launcher Activity. The **data payload** is still delivered to `onMessageReceived()` when the app next opens.

Best practice: use data-only messages and always build notifications in `onMessageReceived()` for consistent behaviour in both states.

57 SharedPreferences internals? Limitations?

Stores data in an **XML file** on disk under `data/data/<package>/shared_prefs/`. **apply()**: writes asynchronously (fast, recommended). **commit()**: writes synchronously, returns boolean. **Limitations**: not suitable for large/complex data, not encrypted by default (use `EncryptedSharedPreferences`), not safe for complex read-modify-write operations across threads, and not a replacement for Room for relational data.

58 Retrofit vs Volley?

Retrofit: type-safe HTTP client using annotated interface definitions. Supports RxJava, Coroutines, and Gson/Moshi converters out of the box. Clean API definition separated from implementation. **Volley**: Google's older request-queue based library, built-in image loading, but no type safety and difficult to integrate with RxJava. **Retrofit is universally preferred** for structured API calls today.

59 Glide / Picasso? Caching and memory handling?

Glide: supports GIF, video frames, more configurable, smaller decoded image size (RGB_565 default), integrates with Activity/Fragment lifecycle to auto-cancel requests. **Picasso**: simpler API but uses ARGB_8888 (higher memory). Both use a **2-level cache**: memory cache (LRU in RAM) and disk cache. Requests are automatically cancelled when the associated lifecycle ends — preventing stale callbacks to destroyed UI.

60 Parcelable internals? What if a Serializable class member is not serializable?

Parcelable is Android's custom serialisation interface — faster than Java Serializable because it does NOT use reflection. You manually write fields to and read from a Parcel (flat binary representation). If a member of a Serializable class is not serializable, a **NotSerializableException** is thrown at runtime. Fix: mark the field **transient** (it will be null after deserialization) or make that type implement Serializable.

61 Different screen sizes/densities/languages/orientations? RecyclerView internals?

Use **dp** for dimensions, **sp** for text, resource qualifiers (layout-sw600dp, values-night, values-fr), ConstraintLayout for flexible layouts, and percentages for proportional sizing.

RecyclerView uses a **RecycledViewPool** — a scrap heap of recycled ViewHolders by view type. When a view scrolls off screen it enters the pool. When a new view is needed, one is taken from the pool and rebound instead of inflated fresh — this is the core performance mechanism.

62 GSON vs Jackson vs Moshi — trade-offs?

Library	Speed	Kotlin Support	Android Fit	Notes
GSON	Slow (reflection)	Poor	Good	Most common, simple
Jackson	Fastest	Via module	Heavy	Feature-rich, large dep
Moshi	Fast (codegen)	Excellent	Best	Modern, null-safe, small

For Java + Android: **Gson** is most common. **Moshi** recommended for new projects — uses code generation to avoid reflection, null-safe, smaller footprint.

EXPERT — Android Advanced & Architecture (Q63 – Q77)

63 Clean Architecture in Android? Three layers? Dependency Rule?

Presentation: ViewModel, Fragments — UI state and user events only. **Domain**: UseCases, Repository interfaces, Domain Models — pure Java, **zero Android imports**, business logic here. **Data**: Repository implementations, DTOs, mappers, Retrofit, Room — implements domain interfaces.

Dependency Rule: dependencies only point **inward**. Presentation depends on Domain. Data depends on Domain. Domain knows nothing about Presentation or Data. This means Domain can be unit tested on the JVM without any Android SDK.

64 MVVM vs MVP vs MVC? Why MVVM preferred? What does MVP's passive View solve?

MVC: Controller (Activity) handles both business logic and UI — fat Activity, hard to test. **MVP:** Presenter has no Android dependency, View is a passive interface — Presenter is testable on JVM. Problem: boilerplate View interfaces. **MVVM:** ViewModel doesn't reference the View at all — LiveData pushes data, View observes. Survives rotation. Less boilerplate than MVP. MVVM is preferred because ViewModel lifecycle awareness eliminates the manual lifecycle management that MVP requires.

65 SavedStateHandle vs onSaveInstanceState()? When does ViewModel not survive but SavedStateHandle does?

onSaveInstanceState(): OS-managed Bundle that survives both rotation AND process death. **ViewModel:** survives rotation but NOT process death (killed when user swipes away in recents). **SavedStateHandle:** injected into ViewModel, backed by onSaveInstanceState() — survives process death. When the OS kills the process, SavedStateHandle data is persisted by the OS and restored when the app relaunches, even though the ViewModel itself is gone.

66 Dagger2 internals? @Component, @Module, @Provides, @Inject?

Dagger2 is a **compile-time** DI framework — generates Java source code, no reflection at runtime. **@Inject** on a constructor: tells Dagger how to create that type. **@Module:** a class providing bindings for types Dagger cannot construct directly (third-party classes). **@Provides:** a method in a Module returning the dependency. **@Component:** the bridge — knows all Modules and satisfies all @Inject requests. Dagger generates a concrete **DaggerXxxComponent** implementation class that wires everything.

67 Scope in Dagger2/Hilt? @Singleton, @ActivityScoped, @ViewModelScoped? Unscoped?

Scope controls the lifetime of a binding — how long the same instance is reused. **@Singleton:** one instance for the entire app lifetime (SingletonComponent). **@ActivityScoped:** one instance per Activity (ActivityComponent). **@ViewModelScoped:** one instance per ViewModel (ViewModelComponent). **Unscoped:** new instance created every injection request — no shared state. Common bug: injecting an unscoped Repository into two ViewModels gives two separate instances — they don't share state.

68 volatile vs transient in Java? When is volatile insufficient?

volatile: guarantees **visibility** — writes immediately visible to all threads. Does NOT guarantee atomicity for compound operations like count++ (read-modify-write). For that, use **AtomicInteger** or **synchronized**.

transient: marks a field to be **excluded from Java serialization** — field is skipped and set to default value on deserialization. Completely unrelated to threading. Use case: don't serialise passwords, computed caches, or non-serialisable objects.

69 What causes the memory leak in AsyncTask?

A non-static inner AsyncTask holds an **implicit reference to the outer Activity**. If the Activity is destroyed (rotation, back press) before doInBackground() finishes, the AsyncTask still references the dead Activity — preventing GC. onPostExecute() may then try to update destroyed UI causing crashes. **Fix:** use a static inner class + WeakReference<Activity>, or better — use RxJava/ViewModel which handle lifecycle correctly.

70 RxJava3 Schedulers internals? subscribeOn() vs observeOn()? Calling subscribeOn() twice?

subscribeOn(): sets the thread for the **source emission and all upstream operators**. Only the **first call** in the chain takes effect — subsequent calls are ignored. **observeOn():** switches the thread for all **downstream operators** from that point forward — can be called multiple times mid-chain.

Operators placed between subscribeOn(io()) and observeOn(mainThread()) run on **io()**. Operators after observeOn(mainThread()) run on the **main thread**. Calling subscribeOn() twice: only the first wins.

71 Backpressure? Does Observable support it? Flowable? Strategies?

Backpressure: the upstream producer emits faster than the downstream consumer can process. **Observable does NOT support backpressure.** Use **Flowable** (implements Reactive Streams spec with `Subscription.request(n)`).

Backpressure strategies: **BUFFER** (buffer all — risk `OutOfMemoryError`), **DROP** (discard items consumer can't handle — good for UI events), **LATEST** (keep only most recent — good for sensor/location), **ERROR** (throw `MissingBackpressureException`), **MISSING** (delegate to downstream).

72 Observable.just(), Observable.fromIterable(), Observable.create() — differences?

just(T...): emits 1–10 fixed items eagerly — items are evaluated immediately at subscription time.
fromIterable(collection): emits each item in a collection one by one. **create(emitter):** full manual control — you call `emitter.onNext()`, `onError()`, `onComplete()` yourself. **create() is lazy** — nothing happens until subscribed. It is the lowest-level way to wrap callbacks or async APIs into an Observable.

73 map() vs flatMap() in RxJava3?

map(): transforms each emitted item **synchronously** — returns a plain value. `Observable<T> → Observable<R>`.
flatMap(): transforms each item into another Observable/Single and **merges results asynchronously**. When the transformation itself is async (returns a Single), `map()` would produce `Observable<Single<R>>` — a nested type that is never subscribed. Only `flatMap()` unwraps it. Scenario: search-as-you-type — use **switchMap** (variant of `flatMap()`) to cancel the previous API call when a new character is typed.

74 RxJava vs Coroutines? Trade-offs for a Java-only project?

Coroutines: Kotlin-only — **not usable in pure Java**. Simpler syntax, more readable, now the standard for Kotlin Android.
RxJava3: works in Java, mature ecosystem, powerful operator library (50+ operators), explicit threading model, steeper learning curve, verbose. For a Java-only project, RxJava3 is the only viable reactive option. If migrating to Kotlin, Coroutines are preferred. Both achieve the same goal — async programming without blocking the main thread.

75 WeakReference in Java? When in Android? Risk?

WeakReference<T> holds a reference that does NOT prevent garbage collection. When the GC runs and only weak references remain, the object is collected and `weakRef.get()` returns **null**. Used in Android to reference Activity/Context in background tasks or caches without causing memory leaks. **Risk:** `get()` can return null at any time — always null-check before use. Over-relying on WeakReference instead of proper lifecycle management (`ViewModel`, `CompositeDisposable`) is an anti-pattern.

76 Android App Bundle (AAB) and Dynamic Feature delivery?

An **AAB** is a publishing format containing all code and resources. Google Play uses it to generate **optimised APKs per device** — only the code, resources, language, and ABI that device needs are downloaded, reducing app size by 15–20%. **Dynamic Feature modules** allow features to be downloaded on-demand using the **Play Core library** (`SplitInstallManager`). The user downloads the base app and requests additional modules only when needed — e.g., an AR feature downloaded only when first used.

77 ProGuard vs R8? Extra optimisations R8 provides?

ProGuard: classic Java bytecode optimiser — three separate passes (shrink → obfuscate → optimise). **R8:** Google's replacement (default since AGP 3.4) — performs all three in a **single pass on DEX bytecode**. R8 is faster, produces smaller output, understands Android-specific optimisations (Kotlin null-checks, lambda inlining), and integrates directly into the AGP build pipeline. Both use the same `proguard-rules.pro` configuration — migrating to R8 requires no rule changes.

This answer sheet covers all 77 questions that were NOT strikethrough in the viva document. OOP: Q1–Q16 (Easy + Medium only — all Expert OOP were strikethrough). Android: Q17–Q77 (Easy + Medium + Expert). Answers are written at the depth expected for an intermediate-to-expert Java Android developer.